



Universität Hamburg
DER FORSCHUNG | DER LEHRE | DER BILDUNG



EMOTIONAL LEARNING
AND MEMORY
GRK / RTG 2753

fMRIPrep Workshop

From raw DICOM files to preprocessed fMRI data

Fabian Dorok & Sterre van de Langenberg

Department of Cognitive Psychology
University of Hamburg

Fabian.dorok@uni-hamburg.de

Sterre.van.de.langenberg@unihamburg.de

Research Training Group “Emotional Learning and Memory”

Workshop overview

Day 1

From DICOM to your 1st subject

- Why fMRIPrep?
- Pipeline anatomy
- Requirements
- BIDS
- Preprocessing your first subject
- Output structure / QC

Day 2

Parallel processing and next steps

- Recap
- Parallelization
- Next-step tools
- Caveats and concluding remarks
- Resources

Day 1: detailed schedule

09:00 - 10:15 | Introduction

- ★ About this workshop
- ★ Why fMRIPrep?
- ★ Pipeline anatomy
- ★ Requirements
- ★ Getting started (WSL, Docker)

10:15 - 10:30 | Break

10:30 - 11:45 | DICOM to BIDS

- ★ Customizing heuristics templates
- ★ Converting your dataset to BIDS

11:45 - 12:30 | Lunch Break

12:30 - 14:00 | Run fMRIPrep

- ★ Preprocessing your 1st dataset
- ★ Inspecting the output (QC)

fMRIPrep performs **minimal** preprocessing

Why preprocessing?

We need to extract signal (neural activity) from noise and artifacts

- Hardware
- Physics
- Participants

→ corrects known physical and physiological artifacts and aligns images in space/time

Why minimal?

fMRIPrep stops where scientific decisions begin: some steps are analysis-dependent

→ does NOT do smoothing, temporal filtering, confound regression, ICA-based denoising

Why fMRIPrep?

The problem

- Manual pipelines vary across labs
- Inconsistent toolbox choices:



- Hard to reproduce
- Error-prone for non-experts
- Time-consuming to build or maintain

fMRIPrep's solution

Combining tools into one “superhero”

- Leverages “the best tool” for each processing step
- Works on *any* data
- Standardized
→ Transparency + Reproducibility
- Easy to use
- Collaborative

What do we want to provide through this workshop?

1. **Lower the barrier to entry**

Hands-on guidance so you get started with confidence without prior experience in navigating WSL, Docker, and Python

2. **Build practical competence, fast (and avoid annoying errors)**

fMRI preprocessing shouldn't take a week of Googling.
We spent plenty of time troubleshooting so you don't have to.

3. **Reproducible, best-practice workflows**

A standardized pipeline that you can apply to your own data.
(today and for every future study)

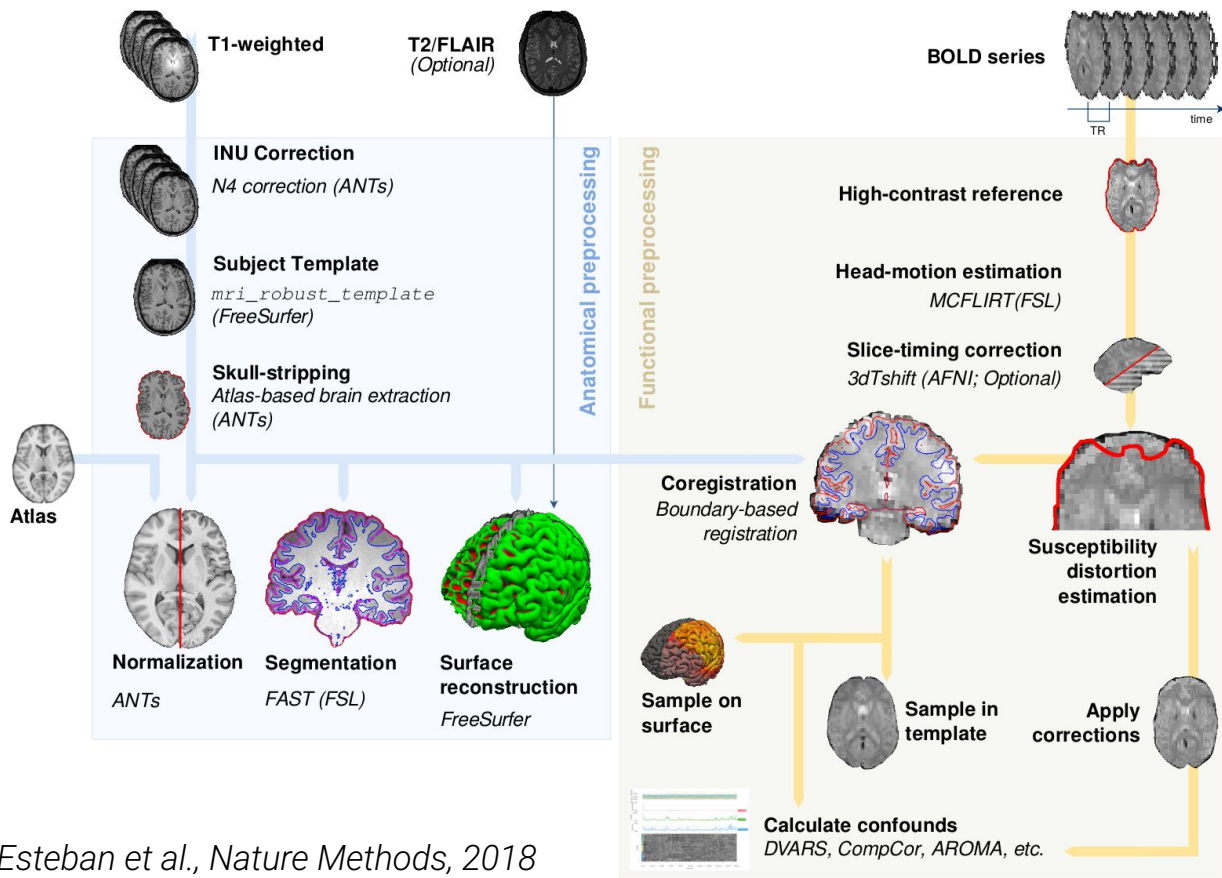
4. **Lots of resources!** (slides, notebook tutorial, support, etc.)

Accessible via [Github](#)



5. Lots of memes ...

How fMRIPrep works: pipeline anatomy



nature methods

ARTICLES

<https://doi.org/10.1038/s41592-018-0235-4>

fMRIPrep: a robust preprocessing pipeline for functional MRI

Oscar Esteban^{1*}, Christopher J. Markiewicz², Ross W. Blair³, Craig A. Moodie⁴, A. Ilkay Isik⁵, Asier Erramuzpe⁶, James D. Kent⁴, Mathias Goncalves⁵, Elizabeth DuPre⁵, Madeleine Snyder⁷, Hiroyuki Oya⁸, Satrajit S. Ghosh^{9,10}, Jesse Wright¹, Joke Durnez¹¹, Russell A. Poldrack¹² and Krzysztof J. Gorgolewski^{13*}

- Modular design
- Self-adjusts automatically
- Reproducible

→ Every step is documented, software versions are recorded, and the HTML report lets you visually check the data

How fMRIPrep works: software tools (Nipype)

ARTICLES

NATURE METHODS

Table 1 | State-of-the-art neuroimaging offers a catalog of readily available software tools

Preprocessing task	Included with fMRIPrep	Alternatives (not included in fMRIPrep)
Anatomical T1-weighted brain extraction	antsBrainExtraction.sh (ANTs)	bet (FSL), 3dSkullstrip (AFNI), MRTTool (SPM plug-in)
Anatomical surface reconstruction	recon-all (FreeSurfer)	CIVET, BrainSuite, Computational Anatomy (SPM plug-in)
Head-motion estimation (and correction)	MCFLIRT (FSL)	3dvolreg (AFNI), spm_realign (SPM), cross_realign_4dof (4dof), antsBrainRegistration (ANTs)
Susceptibility-derived distortion estimation (and unwarping)	3dqwarf (AFNI)	FUGUE and topup (FSL); FieldMap and HySCO (SPM plug-ins)
Slice-timing correction	3dTshift (AFNI)	slicetimer (FSL), spm_slice_timing (SPM), interp_4dof (4dof)
Intrasubject registration	bbregister (FreeSurfer), FLIRT (FSL)	3dvolreg (AFNI), antsRegistration (ANTs), Coregister (SPM GUI)
Spatial normalization (intersubject co-registration)	antsRegistration (ANTs)	@auto_t1rc (AFNI), FNIRT (FSL), Normalize (SPM GUI)
Surface sampling	mri_vol2surf (FreeSurfer)	SUMA (AFNI), MNE, Nilearn
Subspace projection denoising (e.g., independent or principal component analysis)	MELODIC (FSL), ICA-AROMA	Nilearn, LMGS (SPM plug-in)
Confounds	In-house implementation	fsl_motion_outliers (FSL), TAPAS PhysIO (SPM plug-in)
Detection of non-steady states	In-house implementation	Ad hoc implementations, manual setting

fMRIPrep integrates best-in-breed tools for each of the preprocessing tasks that its workflow covers, except for steps implemented as part of the development of fMRIPrep (in-house implementations). Tasks listed in the first column are described in detail in Supplementary Note 1.

Requirements: *FreeSurfer* license file



fMRIPrep uses FreeSurfer tools, which require a license for use.

To download your copy of *FreeSurfer*, please go to:
<https://surfer.nmr.mgh.harvard.edu/fswiki/DownloadAndInstall>

While a registration key is not required to download and install *FreeSurfer*, a license.txt key file is necessary to make the software fully operational.

Please provide the following information to register your copy of *FreeSurfer* and obtain your license.txt key file:

Your name:
Title:
First:
Last:

Name of your academic/research institution:
Institution:

Type of institution which will use this software:
☒ Non-profit education/research
☐ Medical
☐ Commercial

Your e-mail address:
e-mail: (WARNING: email addresses in the qq.com domain do not work as our emails to that domain never get delivered)

Your mailing address:
Address1:
Address2:
City:
State/Province:
Zip/Postal Code:
Country: (Select your location)
Telephone:
Fax:

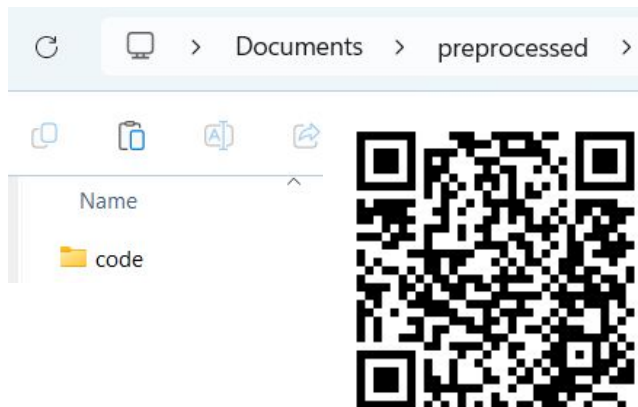
Operating System/Platform:
☐ Linux/Intel
☒ Darwin/Mac

Individual/Multiuser Install:
If you are installing this software to use just for yourself, leave the following input set to 1. However, if you are installing the software for multiple users to run, please give an estimate of the number of users.
Number of Users:

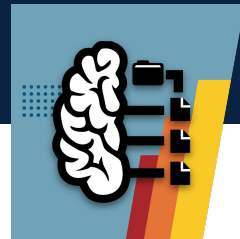
Go to the link below to register:

<https://surfer.nmr.mgh.harvard.edu/registration.html>

& move the license to:



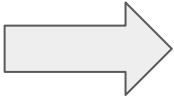
Requirements: the BIDS standard



Why does the Brain Imaging Data Structure matter?

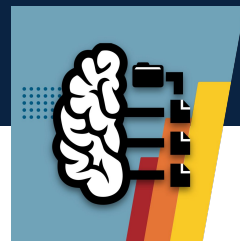
- Anyone (any tool) can understand (and use!) your dataset without documentation
- Automatic metadata → critical for fMRIPrep's functionalities (distortion correction)
- Helps with data validation (BIDS validator)

```
MRI_DATA_2024_03_14/  
  0001_localizer/  
    IM-0001-0001.dcm  
    IM-0001-0002.dcm  ...  
  0002_T1_MPRAGE_1mm/  
    IM-0002-0001.dcm  ...  
  0003_ep2d_bold_rest_p2/  
    IM-0003-0001.dcm  ...  
  0004_ep2d_bold_rest_p2_SBRef/  
    IM-0004-0001.dcm  ...
```



```
my_study/  
  dataset_description.json  
  participants.tsv  
  sub-01/  
    anat/  
      sub-01_T1w.nii.gz  
      sub-01_T1w.json  
    func/  
      sub-01_task-rest_bold.nii.gz  
      sub-01_task-rest_bold.json
```

Requirements: the BIDS standard



Why does the Brain Imaging Data Structure matter?

bioRxiv [Preprint]. 2026 Feb 5:2026.02.03.703514. doi: 10.64898/2026.02.03.703514.

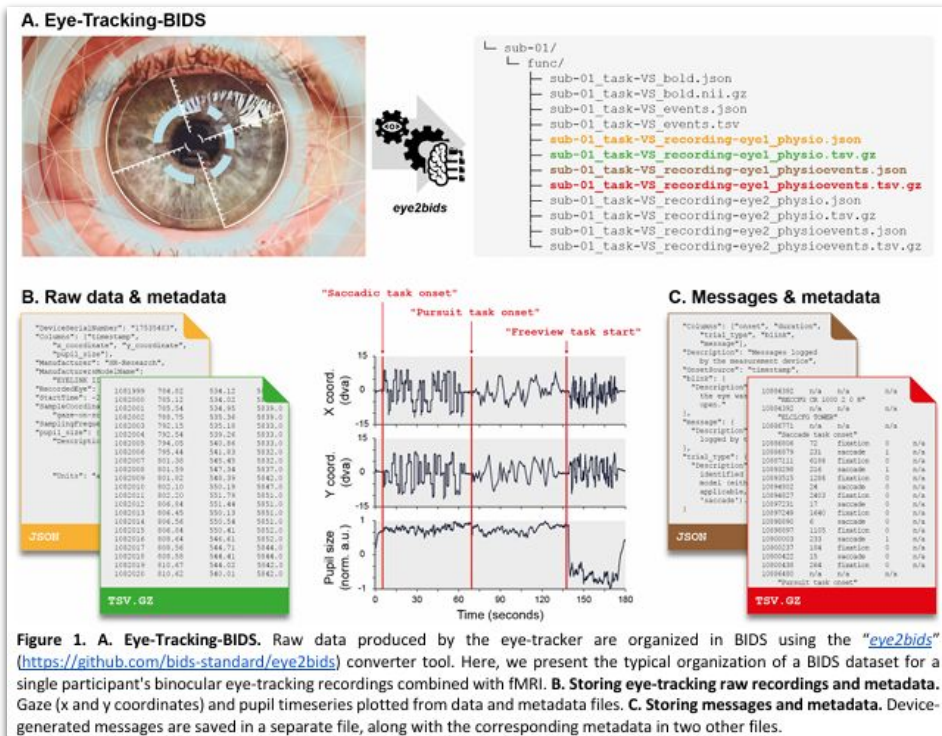
Eye-Tracking-BIDS: the Brain Imaging Data Structure extended to gaze position and pupil data



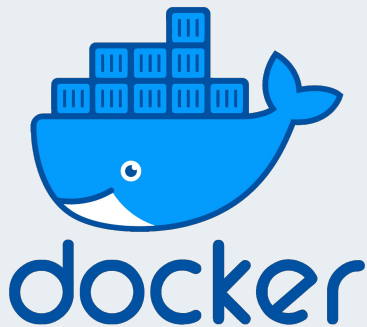
Martin Szinte ¹, Dominik R Bach ², Dejan Draschkow ³, Oscar Esteban ⁴, Benjamin Gagl ⁵, Rémi Gau ⁶, Klara Gregorova ⁷, Yaroslav O Halchenko ⁸, Scott Huberty ⁹, Sina M Kling ¹, Sourav Kulkarni ², BIDS Maintainers; Christopher J Markiewicz ¹¹, Mark Mikkelsen ¹², Robert Oostenveld ¹³, Julia-Katharina Pfarr ⁶

It is no longer only about *brain* data!

- Transparency
- Reliability of results



Requirements: a working environment for fMRIPrep



```
docker run nipreps/fmriprep:latest ...
```

- ★ Easy setup, auto image pull → best for local machine/laptop
- ★ Needs root; blocked on most HPCs



```
Singularity run fmriprep.sif
```

- ★ No root needed → best for HPC clusters
- ★ Build image separately

Getting started (1): Retrieve course materials



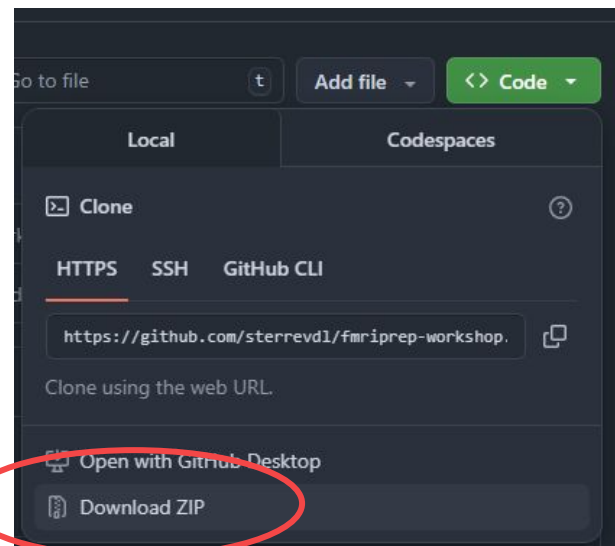
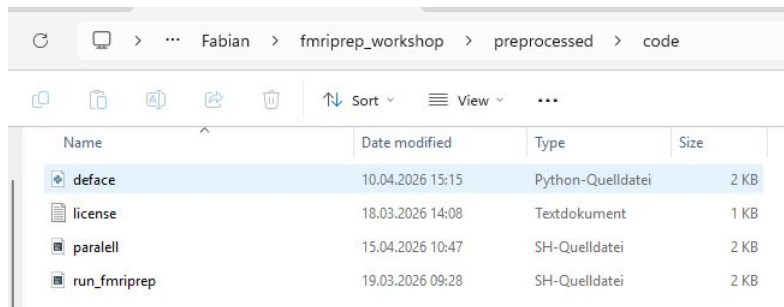
Get all slides, code (notebook tutorial) and the dummy dataset via GitHub:

<https://github.com/sterrevdl/fmripreg-workshop>

Click the green **<> Code** button

→ Download ZIP and unzip it somewhere easy to find
(e.g., /Documents/or under C: if you want to code less later)

Inside your project folder (e.g., fmripreg-workshop),
Create/move your “preprocessed” subfolder, where you are
Storing the FreeSurfer [licence.txt](#) file in /code/



Getting started (2): Install WSL (*Windows users only*)

Docker runs on *Linux*. Windows users therefore first need to set up a Windows Subsystem for Linux (**WSL**). Mac runs on *Unix* natively, so Mac users can take a brief nap.

Open up Windows Powershell as admin. Type this:

```
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart
```

```
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
```

Now, restart your PC. Then open Powershell again as admin.

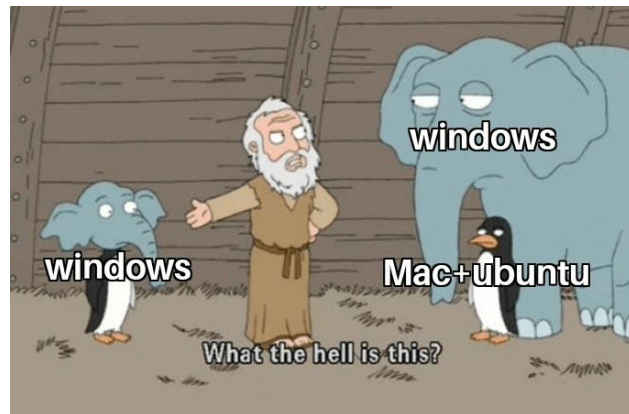
Type this: `wsl --install -d Ubuntu`

Set a username and password.

You should now see something like this:

```
vdldadmin@NBC-L-057727:/mnt/c/WINDOWS/system32$
```

Then `exit` to close WSL.



Getting started (2): Install WSL (*Windows users only*)

If everything worked correctly, you should now be able to run:

```
wsl --list --verbose
```

You should see this:

NAME	STATE	VERSION
* Ubuntu	Running	2

Now we are ready to install Docker within our wsl!

PS: Mac Users, time to wake up.



Getting started (3): Install VS Code

Install Visual Studio Code: <https://code.visualstudio.com/>

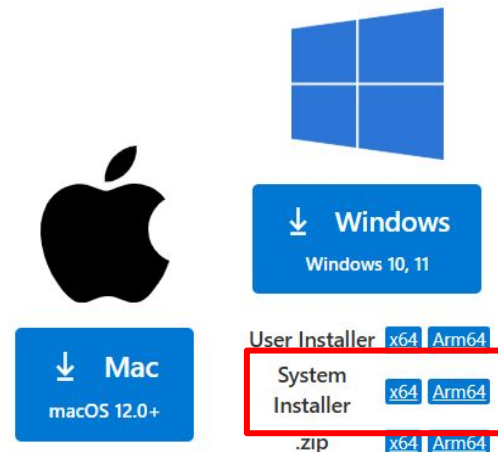
VS code is a free lightweight code editor that will be very helpful for running future neuroimaging analyses with Python-based pipelines.

In this workshop we'll mainly use it for customizing or files (.py, .sh).

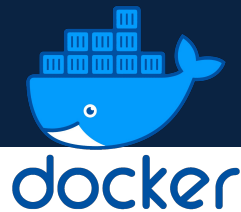
Open VS Code and install: Python, WSL (Ctrl+Shift+x)

NOTE: If VS code is not yet installed:

1. Install Windows or Mac VS Code (from <https://code.visualstudio.com/>)
2. Open VS Code → Extensions → search “WSL” → Install
3. Now open the WSL terminal and type “code .”



Hands-on #1 - Installing Docker



Windows:

Open `ws1` → You should now see something like:

```
sterrevdl@NBC-L-057727: /mnt/c/WINDOWS/System32$
```

Run the following commands to install Docker:

```
# 1) Windows (WSL) Setup
```

```
sudo apt update
```

```
sudo apt install -y ca-certificates curl
```

```
# 2) Add Docker GPG key
```

```
sudo install -m 0755 -d /etc/apt/keyrings
```

```
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \-o /etc/apt/keyrings/docker.asc
```

```
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

```
# continues on the next page ...
```

Mac:

Open the Terminal.

```
....
```

```
....
```

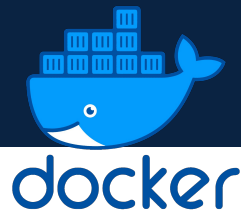
```
# 1) Mac Setup (this is the only command)
```

```
brew install colima docker
```

```
Colima start
```

More details on Docker's website [here](#).

Hands-on #1 - Installing Docker



3) Add Docker repo

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

4) Install Docker

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
docker-buildx-plugin docker-compose-plugin
```

5) Add docker to admin group

```
sudo groupadd docker
sudo usermod -aG docker $USER
newgrp docker
```

Restart wsl

Hands-on #2 - Allocate memory to WSL (Windows users)

Since we are on local laptops and this pipeline is very resource heavy, we will need to allocate more memory to the wsl. Else, it will crash.

First, check how many RAM your device has.
Search for system information

System Information

File Edit View Help

System Summary

- Hardware Resources
- Components
- Software Environment

Locale	United States
Hardware Abstraction Layer	Version = "10.0.20348.4647"
User Name	Not Available
Time Zone	W. Europe Standard Time
Installed Physical Memory (RAM)	1,00 TB
Total Physical Memory	1,00 TB
Available Physical Memory	898 GB
Total Virtual Memory	1,05 TB
Available Virtual Memory	918 GB
Page File Space	50,0 GB
Page File	C:\pagefile.sys
Kernel DMA Protection	Off
Virtualization-based security	Running



Hands-on #2 - Allocate memory to WSL (Windows users)

Open Windows powershell. If still open, **exit** WSL.

```
cd C:\Users\<Your_username>
```

```
notepad .wslconfig
```

Text editor will pop up (agree to create a new file)

Write this inside:

```
[wsl2]
```

```
memory=7GB
```

```
processors=4
```

```
swap=2GB
```

→ fMRIPrep recommends at least 8GB memory. Do not allocate your entire memory to wsl. Set to 8gb if you have more than 8GB (or otherwise n-1).

Save and close. Now restart your wsl:

```
wsl --shutdown
```

Then start it again and check how much memory is available - We should be all set!

```
wsl
```

```
free -h
```

```
free bbc7918@MW-S-KOPSY02:/mnt/c/Users/bbc7918$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	786Gi	7.5Gi	784Gi	3.6Mi	446Mi	779Gi
Swap:	8.0Gi	0B	8.0Gi			

Notepad



Cannot find the .wslconfig file.

Do you want to create a new file?

Yes

No

Cancel

Intermezzo - Mounting folders

```
sudo mkdir -p /mnt/uhhdisk
```

```
sudo mount -t drvfs '\\uhhdisk.uni-hamburg.de\daten\pb' /mnt/uhhdisk -o username=bbc7918,password="xxx"
```

```
# Open up wsl.conf
```

```
sudo nano /etc/wsl.conf
```

```
# Enter this
```

```
[automount]
```

```
enabled = true
```

```
mountFsTab = true
```

```
# Open fstab
```

```
sudo nano /etc/fstab
```

```
# Enter this
```

```
\\uhhdisk.uni-hamburg.de\daten\pb /mnt/uhhdisk drvfs username=your b-kennung,password=your password 0 0
```

```
wslsudo
```

Coffee time!



BIDS - How to get there?

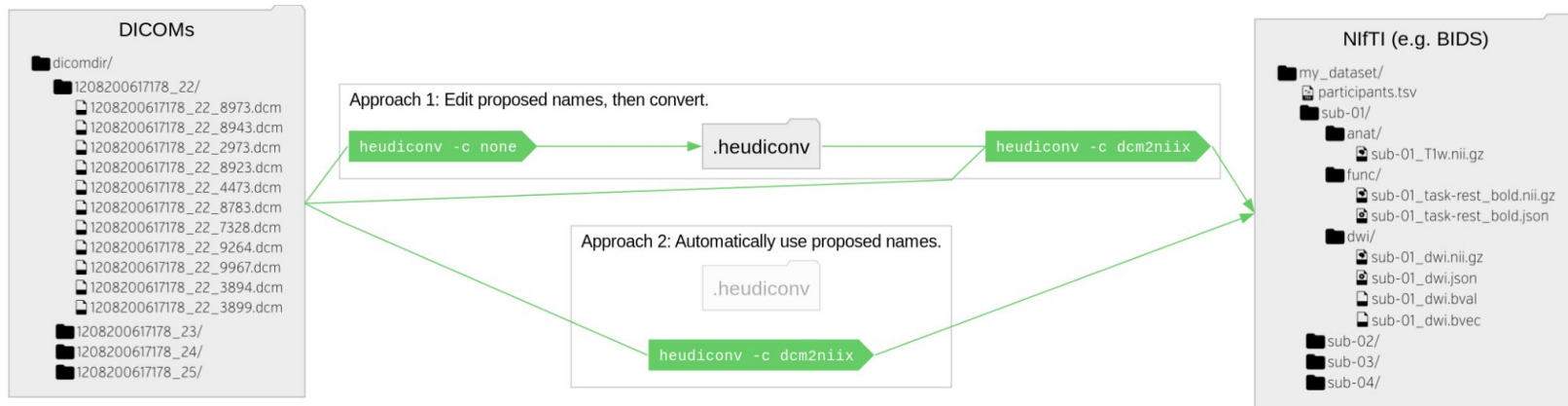
Both use dcm2niix under the hood, but they differ in how you tell them to name things

(1) Dcm2bids

- Config via JSON config file
- Moderate flexibility
- Gentle learning curve
- Suited for single-site/simple setups
- “Series Description matching”
- <https://unfmontreal.github.io/Dcm2Bids/3.1.1/>

(2) HeuDiConv

- Config via Python heuristic file
- Very high flexibility
- Steeper learning curve
- Suited for complex/multi-site setups
- “infotodict() function”
- <https://heudiconv.readthedocs.io/en/latest/>



Hands-on #3 - Heu(ristic) for Di(com) Conv(ersion)

We're using **HeuDiConv**!

Install HeuDiConv in docker:

```
docker pull nipy/heudiconv:latest
```

We will now run HeuDiConv without any conversion to create a template for the heuristics file.

First, go to the location of your dataset (e.g., the example data you downloaded via Github).
For example, you might have saved it here: C://fmriprip_workshop/Datasets

Type:

```
cd /mnt/c/fmriprip_workshop/Datasets
```

The first slash at /mnt is required!

Note: Windows drives (like C :) are automatically available under /mnt/. However, shared network drives (like uhhdisk) are not always mounted automatically. We explain how to do this drives in the notebook.

Hands-on #4 - Understanding your data structure

Please try to understand the structure of the dataset:

- How many subjects, sessions and tasks were included in this study?
- What is the order in which the scanning sequences were taken?

```
sub-098
├── EP2D_BOLD_RESTING_STATE_0005
├── EP2D_BOLD_RECALL_0009
├── T1_MPRAGE_COR_0016
└── EP2D_BOLD_RESTING_STATE_0007
sub-094
├── EP2D_BOLD_RESTING_STATE_0008
├── T1_MPRAGE_COR_0006
├── EP2D_BOLD_RECALL_0012
└── EP2D_BOLD_RESTING_STATE_0010
sub-012
├── EP2D_BOLD_RESTING_STATE_0006
├── EP2D_BOLD_RESTING_STATE_0008
├── EP2D_BOLD_RECALL_0011
├── EP2D_BOLD_RECALL_0010
└── T1_MPRAGE_COR_0013
```

Note: Interrupted scanning sequences are also saved, creating duplicates: see sub-012 recall.

A higher sequence number indicates the last scan of that sequence, i.e., the complete scan

← view this tree structure by typing:

```
cd C:/fmriprep_workshop/datasets
tree
```

Hands-on #4 - Understanding your data structure

HeuDiConv will extract the task name of the scanning sequences from the folder names (specifically after the `_BOLD_` substring).

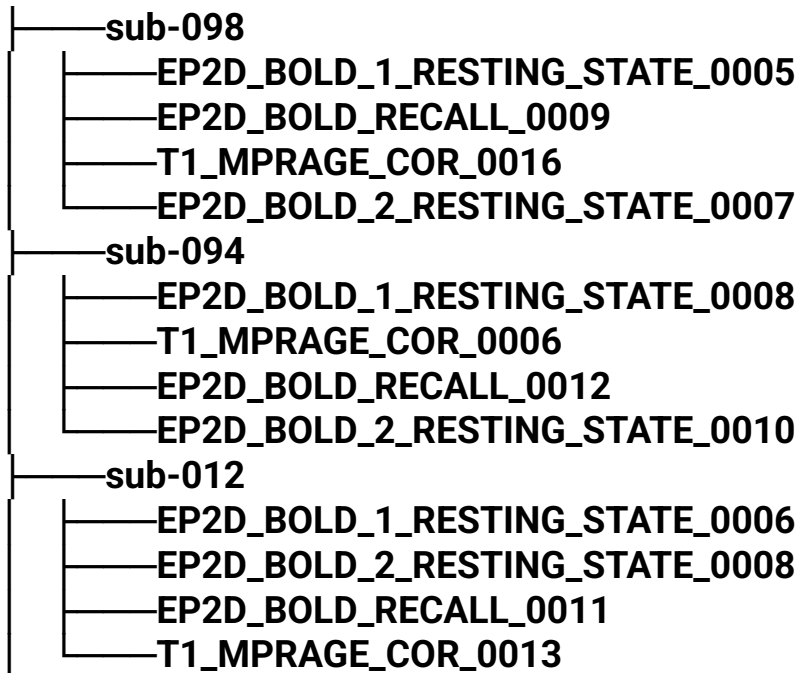
Your task: Prepare the data structure.

1. How would you name the specific folders?
2. What would you do with canceled scanning sequences?

Tip: For the resting state scan, add the run number right between BOLD_ and _RESTING (e.g., EP2D_BOLD_1_RESTING_STATE)

Hands-on #4 - Understanding your data structure

Your final BIDS structure should look something like this:



Hands-on #5 - Create a Heudiconv template

Now run heudiconv without any conversion (-c <<none>>): we're creating a heuristic template.

```
docker run --rm -it \
--user "$(id -u):$(id -g)" \
-v "$PWD:$PWD" \
-w "$PWD" \
nipy/heudiconv:latest \
--files "$PWD"/datasets/sub-098/*/*.IMA \
-o "$PWD"/datasets/Nifti \
-f convertall \
-s 098 \
-c none
```

Flags: heudiconv --files <input DICOMs>

-o <output dir>

-f <heuristic>

-s <subject ID>

-c <converter>.

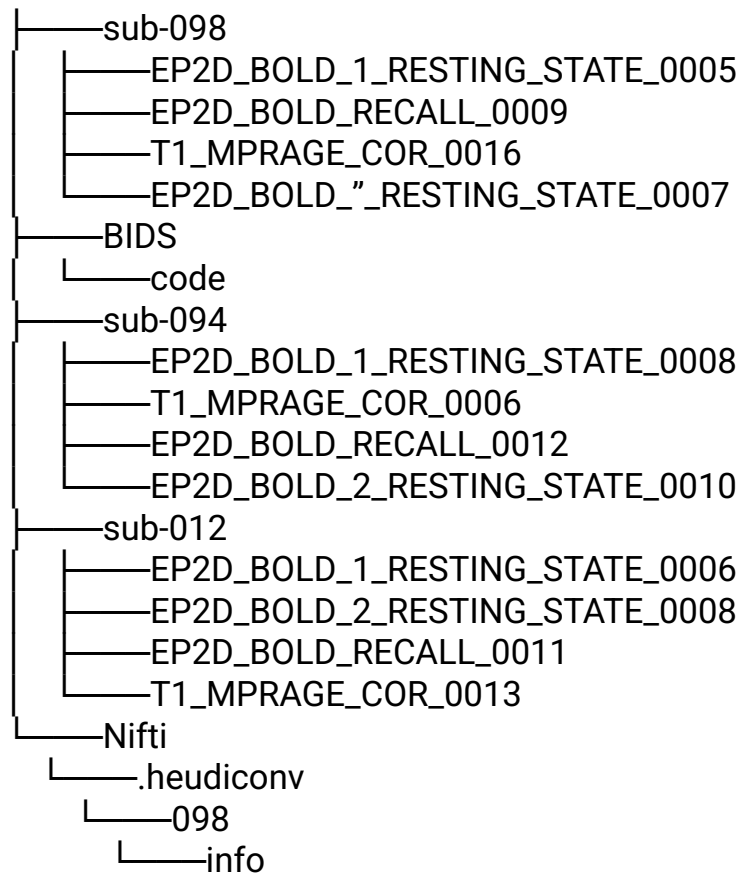
Hands-on #5 - Create a Heudiconv template

Inside your mounted “Datasets” folder,
Docker should have created a folder “Nifti”.

In the Datasets folder, create an additional folder
“BIDS”, with a subfolder “code”.

This is where we will save our [heuristic.py](#) file.

Make sure your folder structure looks like this →



Hands-on #6 - Setting up a venv in VS Code

We will customize our **heuristic.py** in Visual Studio Code.

In File Explorer/Finder, move **heuristic.py** from “.../Nifti/.heudiconv/098/info/” to “../BIDS/code/”

It is recommended to create virtual environments (venvs) for each project you work on to prevent interference!

Hands-on #6 - Setting up a venv in VS Code

How to create a venv in a “quick and easy” way?

First, the wsl terminal

```
sudo apt install python3.12-venv
```

Create venv at the HOME folder of our wsl (indicated by ~).

```
python3 -m venv ~/myfirstvenv
```

Activate it

```
source ~/myfirstvenv/bin/activate
```



Hands-on #7 - Customizing your heuristic.py

Now we are ready to start VS Code.

In your terminal (WSL for Windows), navigate to your BIDS/code folder.

If your Current Directory (cd) is already at “Datasets”, just type: `cd ../BIDS/code`

(Note: `cd ..` let's you move one folder up)

We will now customize the **heuristic.py** to match our dataset structure.

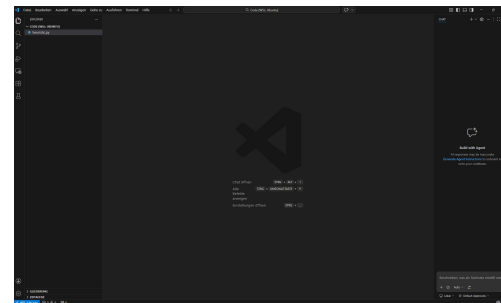
For that we need to open up Visual Studio Code: `code .`

We will customize our **heuristic.py** in Visual Studio Code.

In File Explorer/Finder, move **heuristic.py**
from “../Nifti/.heudiconv/098/info/” to “../BIDS/code/”

NOTE: If VS code is not yet installed:

1. Install Windows or Mac VS Code (from <https://code.visualstudio.com/>)
2. Open VS Code → Extensions → search “**WSL**” → Install
3. Now open the WSL terminal and type “code .”



Hands-on #7 - Customizing the heuristic.py

Double-click heuristic.py.

Also, if prompted on the bottom right, install the python extension for VS Code.

Now we are ready to adjust our heudiconv.py! Double-click in VS Code → Add:

```
import os
```

We will need that later. Delete `from heudiconv.utils import SeqInfo.`

Change

```
def infotodict(  
    seqinfo: list[SeqInfo],) -> dict[tuple[str, tuple[str, ...], None], list[str]]:
```

To

```
def infotodict(seqinfo)
```

N.B. We will only need to make some changes to **infotodict** to customize our heuristic.py.

→ This is where we tell HeuDiConv how to convert the raw files.

Hands-on #7 - Customizing the heuristic.py

Delete the following:

```
"""Heuristic evaluator for determining which runs belong where
```

```
    allowed template fields - follow python string module:
```

```
        item: index within category
```

```
        subject: participant id
```

```
        seqitem: run number during scanning
```

```
        subindex: sub index within group
```

```
    """
```

```
    data = create_key("run{item:03d}")
```

```
    info: dict[tuple[str, tuple[str, ...], None], list[str]] = {data: []}
```

Hands-on #7 - Customizing the heuristic.py

Add `info = {}` so it looks like this:

```
def infotodict(seqinfo):  
    info = {}
```

Delete the comments in the for `s` in `seqinfo` loop:

```
    for s in seqinfo:  
        info[data].append(s.series_id)  
    return info
```

Hands-on #7 - Customizing the heuristic.py

Add this to the bottom of your script:

```
@dataclass
class FakeSeqInfo:
    series_id: int
    dcm_dir_name: str
    protocol_name: str = ""
seqinfo = [
    FakeSeqInfo(series_id=1, dcm_dir_name="T1_MPRAGE_COR_0016"),
    FakeSeqInfo(series_id=2, dcm_dir_name="EP2D_BOLD_RECALL_0009"),
    FakeSeqInfo(series_id=3, dcm_dir_name="EP2D_BOLD_1_RESTING_STATE_0005"),
    FakeSeqInfo(series_id=3, dcm_dir_name="EP2D_BOLD_2_RESTING_STATE_0007")
]
infotodict(seqinfo)
```

For now we're just debugging. Make sure the dcm_dir_name matches your actual directory names!

Hands-on #7 - Customizing the heuristic.py

Your task:

We now need to specify which folder is an anatomical scan and which is a functional scan. For functional scans, we need to specify the task name and, if the task has multiple runs, the run indicator.

How would you do that? (code will be specified as we progress)

Hands-on #7 - Customizing the heuristic.py

Set a breakpoint at `info = {}`

And `infotodict(seqinfo)`

This way, you will see what your code actually does.

HeuDiConv will save our converted data in BIDS format.

The anatomical scan will be saved under your BIDS folder like this

```
sub-{subject}/anat/sub-{subject}_T1w"
```

The functional scans will be saved like this:

```
sub-{subject}/func/sub-{subject}_task-{task}_bold"
```

The curly brackets indicate variables we need to set up. {subject} will be defined directly through HeuDiConv, task will be extracted from the folder names.

Hands-on #7 - Customizing the heuristic.py

For now, add this right after `info = {}`

```
subject = "098"
```

Now we will make use of the folder names saved in `seqinfo` which changes for every MRI folder “s”

```
s.dcm_dir_name
```

Based on the folder name, we will first tell HeuDiConv which folder is t1 and which ones are functional data. For the t1, it is very straightforward. After

```
for s in seqinfo:
```

Add this:

```
    if 'T1' in s.dcm_dir_name:
```

Note that indents matter! This if sentence must be one indent further than `for s in seqinfo:`

To do this, select the line and press tab (if not already automatically indented)

Hands-on #7 - Customizing the heuristic.py

Everything should then look like this:

```
def infotodict(seqinfo):  
    info = {}  
    subject = "098"  
    for s in seqinfo:  
        if 'T1' in s.dcm_dir_name:
```

Now we tell HeudiConv how to detect a t1 folder, but we haven't told it what to do with it yet.

This step is very straightforward though:

```
        for s in seqinfo:  
            if 'T1' in s.dcm_dir_name:  
                data = create_key(f'sub-{{subject}}/anat/sub-{{subject}}_T1w')
```


Hands-on #7 - Customizing the heuristic.py

Telling HeuDiConv how to detect functional scans is just as easy:

```
for s in seqinfo:
    if 'T1' in s.dcm_dir_name:
        data = create_key(f'sub-{subject}/anat/sub-{subject}_T1w')
    elif "EP2D" in s.dcm_dir_name:
```

Hands-on #7 - Customizing the heuristic.py

We will first split the folder name into different parts. These parts will tell us if there are multiple runs and if so, which run it is and it will tell us the task name:

```
elif "EP2D" in s.dcm_dir_name:
```

```
    parts = s.dcm_dir_name.split('_')
```

We will now have a list of separate strings like this:

```
['EP2D', 'BOLD', '1', 'RESTING', 'STATE', '0005']
```

The 1 indicates the run number. For recall, it will look like this:

```
['EP2D', 'BOLD', 'RESTING', 'RECALL', '0009']    # no run indicator
```

We can now use this to tell HeuDiConv if multiple runs exist. We will do a simple if sentence:

```
elif "EP2D" in s.dcm_dir_name:
```

```
    parts = s.dcm_dir_name.split('_')
```

```
    if parts[2].isdigit():
```

So we check if the third string (python starts counting from 0) is a digit (i.e., run indicator) or not.

Hands-on #7 - Customizing the heuristic.py

Now we can extract the run number and the task number and finally create our key:

```
elif "EP2D" in s.dcm_dir_name:
    parts = s.dcm_dir_name.split('_')
    if parts[2].isdigit():
        run = parts[2]
        task = parts[3].lower()
        data = create_key(f"sub-{subject}/func/sub-{subject}_task-{task}_run-{run}_bold")
```

We add the .lower() so task is in lower cases (recall) and not in caps (RECALL).

If the third string is not a run indicator, then this scan only had one run. Thus, we can simply write:

```
elif "EP2D" in s.dcm_dir_name:
    parts = s.dcm_dir_name.split('_')
    if parts[2].isdigit():
        run = parts[2]
        task = parts[3].lower()
        data = create_key(f"sub-{subject}/func/sub-{subject}_task-{task}_run-{run}_bold")
    else:
```

Now it's your turn! How would you create the key for scans with only one run?

Hands-on #7 - Customizing the heuristic.py

Solution:

```
elif "EP2D" in s.dcm_dir_name:
    parts = s.dcm_dir_name.split('_')
    if parts[2].isdigit():
        run = parts[2]
        task = parts[3].lower()
        data = create_key(f"sub-{subject}/func/sub-{subject}_task-{task}_run-{run}_bold")
    else:
        task = parts[2].lower()
        data = create_key(f"sub-{subject}/func/sub-{subject}_task-{task}_bold")
```

Now we are almost done with setting up the heuristic!

Add this at the end:

```
if data not in info:
    info[data] = []
    info[data].append(s.series_id)
```

Hands-on #7 - Customizing the heuristic.py

The final def infotodict should look like this:

```
def infotodict(seqinfo):
    info = {}
    subject = "098"
    for s in seqinfo:
        if 'T1' in s.dcm_dir_name:
            data = create_key(f'sub-{subject}/anat/sub-{subject}_T1w')
        elif "EP2D" in s.dcm_dir_name:
            parts = s.dcm_dir_name.split('_')
            if parts[2].isdigit():
                run = parts[2]
                task = parts[3].lower()
                data = create_key(f"sub-{subject}/func/sub-{subject}_task-{task}_run-{run}_bold")
            else:
                task = parts[2].lower()
                data = create_key(f"sub-{subject}/func/sub-{subject}_task-{task}_bold")
        if data not in info:
            info[data] = []
        info[data].append(s.series_id)
    return info
```

Finally, add this at the end of your script:

```
print(info)
```

Hands-on #8 - Running & debugging the heuristic.py

Finally, add this at the end of your script:

```
print(info)
```

Run the script. The output should look like this

```
{('sub-098/anat/sub-098_T1w', ('nii.gz',), None): [1], ('sub-098/func/sub-098_task-recall_bold', ('nii.gz',), None): [2], ('sub-098/func/sub-098_task-resting_run-1_bold', ('nii.gz',), None): [3], ('sub-098/func/sub-098_task-resting_run-2_bold', ('nii.gz',), None): [3]}
```

This means our script is working as intended! Now simply remove all debugging stuff, so all of this:

```
from dataclasses import dataclass
@dataclass
class FakeSeqInfo:
    series_id: int
    dcm_dir_name: str
    protocol_name: str = ""
seqinfo = [
    FakeSeqInfo(series_id=1, dcm_dir_name="T1_MPRAGE_COR_0016"),
    FakeSeqInfo(series_id=2, dcm_dir_name="EP2D_BOLD_RECALL_0009"),
    FakeSeqInfo(series_id=3, dcm_dir_name="EP2D_BOLD_1_RESTING_STATE_0005"),
    FakeSeqInfo(series_id=3, dcm_dir_name="EP2D_BOLD_2_RESTING_STATE_0007")]
info = infotodict(seqinfo)
print(info)
```

Hands-on #8 - Running & debugging the heuristic.py

Finally, change this

```
subject = "098"
```

To this:

```
subject = os.getenv("HEUDICONV_SUBJECT")
```

We are basically telling the script now to extract the subject ID from our HeuDiConv call script.

Setting up the HeuDiConv call script is the next step.

Luckily, we mostly just have to change some paths and that's it!

Hands-on #9 - Calling heudiconv via a bash script

Download and install [notepad++](#)

Copy this code in there:



```
#!/bin/bash
# Directory containing the data (modify this to your actual path)
data_dir="/mnt/c/fmriprep_workshop/Datasets"

# Loop over each directory that matches the pattern
for subject_session_dir in "$data_dir"/sub-*; do
    # Extract the subject from the folder name
    subject=$(basename "$subject_session_dir" | sed -E 's/^sub-([0-9]{3})$/\1/')

    echo "Processing subject: $subject"
    # Run the Docker command with the corresponding environment variables
    docker run --rm \
        -v "/mnt/c/fmriprep_workshop/Datasets:/data" \
        -v "/mnt/c/fmriprep_workshop/Datasets/BIDS/code:/heuristics" \
        -v "/mnt/c/fmriprep_workshop/Datasets/BIDS:/outdir" \
        --entrypoint "heudiconv" \
        -e HEUDICONV_SUBJECT=$subject \
        nipy/heudiconv:latest \
        -d "/data/sub-{subject}/*/.*.IMA" \
        -s $subject \
        -o "/outdir" \
        -f "/heuristics/heuristic.py" \
        --overwrite --bids -c dcm2niix
done
```


Hands-on #9 - Calling heudiconv via a bash script

Adjust your paths accordingly. If your raw subject data folders are located at C:/fmriprip_workshop/datasets, type

```
data_dir="/mnt/c/fmriprip_workshop/datasets"
```

Make sure your raw subject data folders are called sub-XXX. HeuDiConv will loop over those.

Furthermore, we extract the subject ID from there using

```
subject=$(basename "$subject_session_dir" | sed -E 's/^sub-([0-9]{3})$/\1/')
```

This is actually the {subject} variable the heuristic file will use.

What we do then is simply specify the parts we need.

```
-v "/mnt/c/fmriprip_workshop/Datasets:/data" \
```

Is the location where it will look for your .IMA files. The ./data is just a shortcut for the whole path.

For the -d (DICOM) flag, it will then search for the .IMAs under

```
d "/data/sub-{subject}/**/*.IMA" \
```

Hands-on #9 - Calling heudiconv via a bash script

We also set the location of our heuristic.py, which should be under ./BIDS/code

```
-v "/mnt/c/fmriprep_workshop/Datasets/BIDS/code:/heuristics"
```

This shortcut is called under the -f (file) flag:

```
f "/heuristics/heuristic.py"
```

Finally, we set an output folder under .Dataset/BIDS

```
-v "/mnt/c/fmriprep_workshop/Datasets/BIDS:/outdir"
```

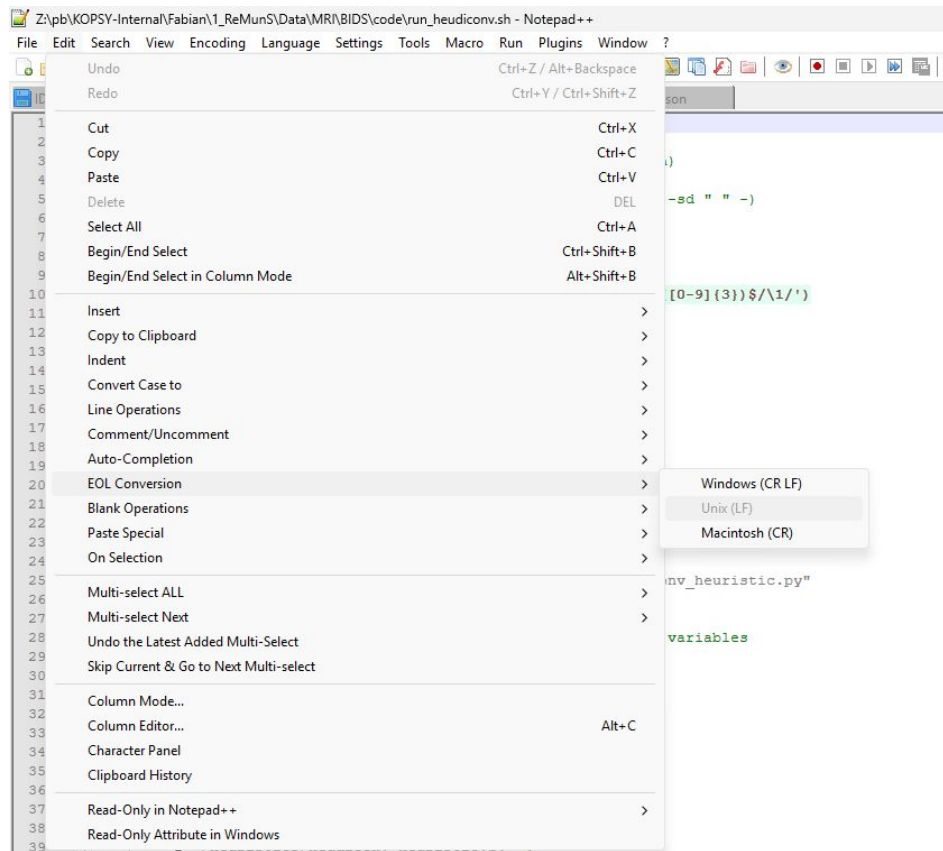
This shortcut is called under the -o (outdir) flag:

```
-o "/outdir"
```

This is basically all there is to it!

Hands-on #9 - Calling heudiconv via a bash script

Under edit, convert the file to Unix (LF)



Hands-on #9 - Calling heudiconv via a bash script

Go to file → Save as. Under “Save as type”, select “Unix script file”.

Name the file run_heudiconv.sh and save it under ./Datasets/BIDS/code.

DELETE /BIDS/.heudiconv !!!!

For windows:

Navigate to this folder in your WSL:

```
cd /mnt/c/fmriprep_workshop/Datasets/BIDS/code
```

Run it:

```
bash run_heudiconv.sh
```

For Mac:

```
chmod +x run_heudiconv.sh
```

Run it:

```
./run_heudiconv.sh
```



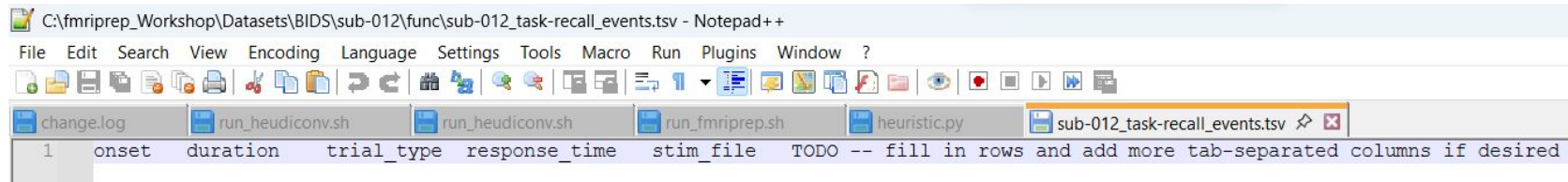
Inspecting the BIDS output: folder structure

Once HeuDIConv finished, your participant folder inside ./BIDS should look like this

```
└──sub-098
    |   sub-098_scans.tsv
    |
    |── anat
    |   sub-098_T1w.json
    |   sub-098_T1w.nii.gz
    |
    |── func
    |   sub-098_task-recall_bold.json
    |   sub-098_task-recall_bold.nii.gz
    |   sub-098_task-recall_events.tsv
    |   sub-098_task-resting_run-1_bold.json
    |   sub-098_task-resting_run-1_bold.nii.gz
    |   sub-098_task-resting_run-1_events.tsv
    |   sub-098_task-resting_run-2_bold.json
    |   sub-098_task-resting_run-2_bold.nii.gz
    |   sub-098_task-resting_run-2_events.tsv
```

Inspecting the BIDS output: events.tsv

Events.tsv will be crucial for your later analyses. It will show when what happened during the scan, such as stimulus presentations, fixation crosses, etc. Right now, it is just an empty template:



```
C:\fmriprep_Workshop\Datasets\BIDS\sub-012\func\sub-012_task-recall_events.tsv - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
change.log run_heudiconv.sh run_heudiconv.sh run_fmriprep.sh heuristic.py sub-012_task-recall_events.tsv
1 onset duration trial_type response_time stim_file TODO -- fill in rows and add more tab-separated columns if desired
```

1	category_onset	category_duration	category
2	8	2	stim
3	10	1	answer
4	11	1	fix
5	12	2	stim
6	14	1	answer
7	15	1	fix
8	16	2	stim
9	18	1	answer
10	19	1	fix
11	20	2	stim

The rest of the files are not that interesting (yet!)

It's basically our raw files converted into single .nii.gz files in BIDS format.

For functional scans, these are 4d files now (x, y, z axes and time).

Ready to run fMRIPrep?! A brief checklist

- ❑ FreeSurfer license available
- ❑ Docker installed
- ❑ DICOMs organized by subject
- ❑ Full HeuDiConv run completed for all subjects
- ❑ The BIDS output looks OK and contains all relevant scans
(bids-validator shows 0 errors // warnings OK)
- ❑ participants.tsv has all subjects
- ❑ Sufficient storage for fMRIPrep outputs (~8 GB/subject)
→ make sure to check your PC's resources

Hands-on #10 - Running fMRIPrep on your 1st subject

“Anatomy” of a command:

```
fmriprep /data/bids /data/derivatives/fmriprep participant \  
  --participant-label sub-01 \  
  --fs-license-file /opt/freesurfer/license.txt \  
  --output-spaces MNI152NLin2009cAsym:res-2 fsaverage5 \  
  --nthreads 8 --mem-mb 2500
```

/data/bids	Input: BIDS dataset root directory
/data/derivatives	Output: where will the results saved?
--output-spaces	Which space(s) to normalize to (e.g., template, T1w)
--mem-mb	Choose the amount of memory used
--skip-native	“Don’t output EPI images in MNI space”

Hands-on #10 - Running fMRIPrep on your 1st subject

Good news: Setting up fmriprep is actually super straightforward!

- (1) If not already done, create a new folder <preprocessed> under ./Datasets/
- (2) Add a subfolder <code>.
- (3) Move the freesurfer license here, which we downloaded before ([here](#)).
- (4) Open text editor (e.g., notepad++).
Copy paste the bash script (see also our Notebook).
- (5) Now it's your turn! Adjust the paths accordingly!

```
#!/bin/bash
##### User inputs #####

# BIDS root (as it exists in WSL)
bids_root_dir="your bids path"

# fMRIPrep output directory
output_dir="your preprocessed path"

# FreeSurfer license (adjust if you put it somewhere else)
fs_license="{output_dir}/code/license.txt"

for subj_dir in "$bids_root_dir"/sub-012; do
  [ -d "$subj_dir" ] || continue

  subj=$(basename "$subj_dir" | sed 's/^sub-//')

  echo "=====
  echo "Running fMRIPrep on participant sub-${subj}"
  echo "=====

  docker run --rm \
  -v "$bids_root_dir":/data:ro \
  -v "$output_dir":/out \
  -v "${fs_license}:/opt/freesurfer/license.txt:ro \
  -v /etc/localtime:/etc/localtime:ro \
  --user "${id -u}":"${id -g}" \
  nipreps/fmriprep:latest \
  /data /out participant \
  --participant-label "${subj}" \
  --fs-no-reconall \
  --output-spaces MN1152NLin2009cAsym:res-2 \
  --skip-bids-validation \
  --notrack

  # Cleanup after processing
  echo "Cleaning up Docker data for participant sub-${subj}"
  docker system prune -f --volumes
done
```

Hands-on #10 - Running fmriprep on your 1st subject

Save the script as `run_fmriprep.sh` (convert to unix if needed). Finally, run it in your wsl

```
wsl
```

```
cd /mnt/c/fmriprep_workshop/datasets/preprocessed/code
```

```
bash run_fmriprep.sh
```

Now fmriprep should start preprocessing your data!

Good to know:



--output-spaces	If you want to stay in native space instead of normalizing MNI, write this <code><--output-spaces T1w fsnative fsaverage></code> . Useful for multivariate analyses.
--fs-no-reconall	Remove this flag if you want cortical reconstruction. Recon takes a long time, but will allow you to create anatomical masks in native space. Useful when you stay in native space.

See you tomorrow!

- Parallel processing
- Next steps
- Caveats and concluding remarks

Day 2: schedule overview

09:00 - 09:15 | Recap Day 1 & Running fmriprep command

09:15 - 09:45 | fMRIPrep Outputs

09:45 - 10:15 | Parallelization

10:15 - 10:30 | Coffee break

10:30 - 10:45 | A note on Multi-Session heuristics

10:45 - 12:00 | Next steps (incl Deface)

12:00 - 12:30 | Caveats and concluding remarks

Recap Day 1

DICOM → BIDS → fMRIPrep → Derivatives

↑
heudiconv

↑
Docker

↑
HTML QC

```
#!/bin/bash
##### User inputs #####

# BIDS root (as it exists in WSL)
bids_root_dir="your bids path"

# fMRIPrep output directory
output_dir="your preprocessed path"

# FreeSurfer license (adjust if you put it somewhere else)
fs_license="{output_dir}/code/license.txt"

for subj_dir in "${bids_root_dir}/sub-012"; do
  [ -d "$subj_dir" ] || continue

  subj=$(basename "$subj_dir" | sed 's/^sub-//')

  echo "=====
  echo "Running fMRIPrep on participant sub-${subj}"
  echo "=====

  docker run --rm \
  -v "${bids_root_dir}"/data:ro \
  -v "${output_dir}"/out \
  -v "${fs_license}"/opt/freesurfer/license.txt:ro \
  -v /etc/localtime:/etc/localtime:ro \
  --user "${(id -u)}":"${(id -g)}" \
  nipreps/fmriprep:latest \
  /data /out participant \
  --participant-label "${subj}" \
  --fs-no-reconall \
  --output-spaces MNI152NLin2009cAsym:res-2 \
  --skip-bids-validation \
  --notrack

  # Cleanup after processing
  echo "Cleaning up Docker data for participant sub-${subj}"
  docker system prune -f --volumes
done
```

fMRIPrep output: three broad classes

1. **Visual QA (quality assessment) reports:** one HTML per subject, that allows you to thoroughly inspect the quality of the preprocessing/data (ensures transparency!)
2. **Derivatives (preprocessed data):** this is the data that you will use as input for your analyses (i.e., data after the various preparation procedures have been applied):
 - INU-corrected versions of the T1-weighted image (per subject)
 - the brain mask
 - BOLD images after head-motion correction, slice-timing correction and aligned into the same-subject's T1w space or in some standard space.
3. **Confounds:** this is a special family of derivatives that can be utilized to inform subsequent denoising steps (e.g., movement regressors!)

fMRIprep output (1): Visual QA Reports

Every participant gets an HTML report. Know what to look for:

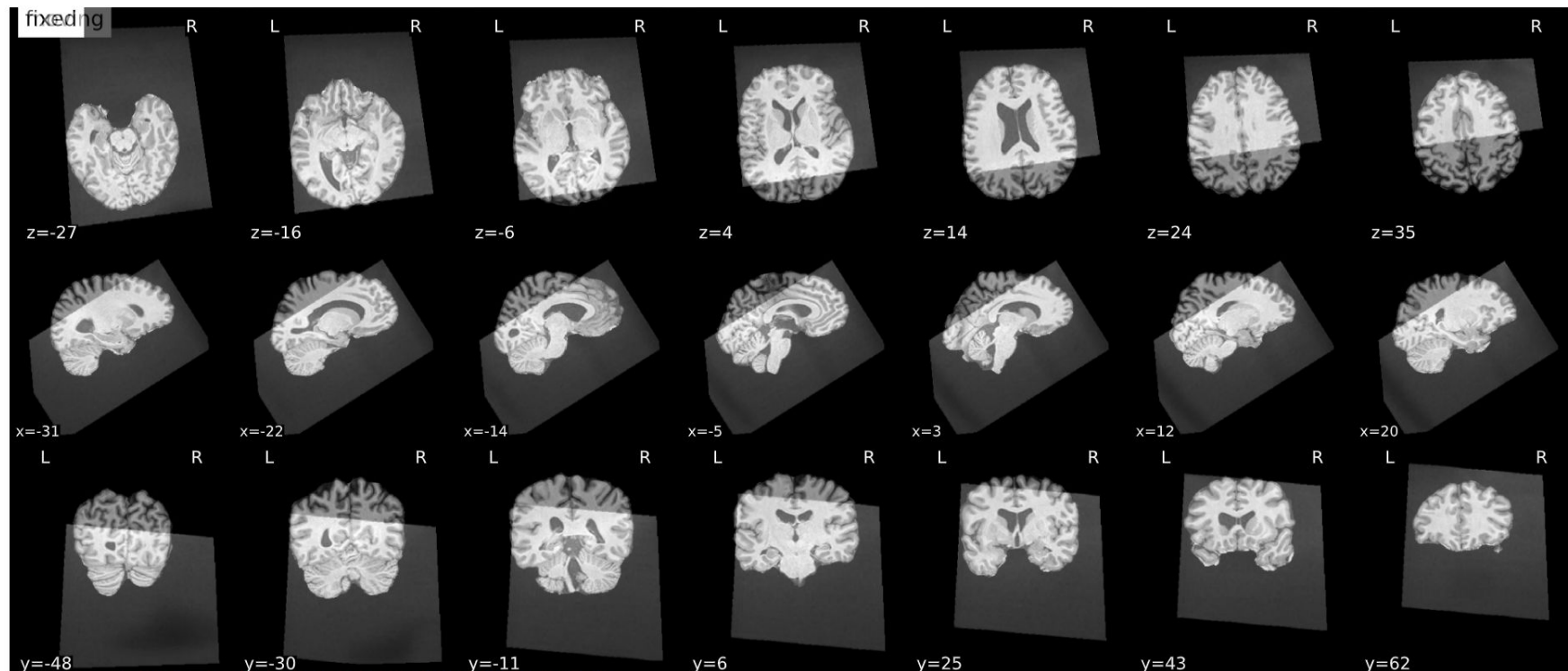
Brain mask	Should tightly cover cortex, not “cut” into it or include too much skull
Tissue segmentation	WM/GM/CSF boundaries should be anatomically plausible
BOLD <> T1w registration	Which template space(s) to normalize to
MNI normalization	Brain shape should match the template (Anterior Commisure)
DVARS & FD plots	Spikes indicate motion (note which volumes may need scrubbing)

`<output_dir>/fmripred/sub-<subject_label>.html`

[View example report](#)

fMRIPrep output (1): Visual QA Reports

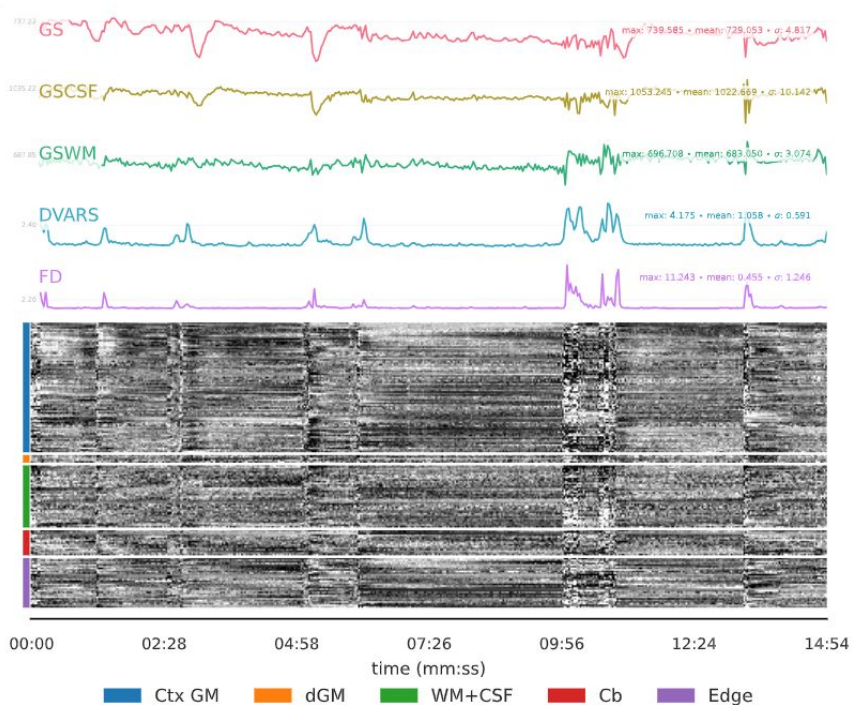
Two functional runs resulted in a complete artifact (scanner door open?)



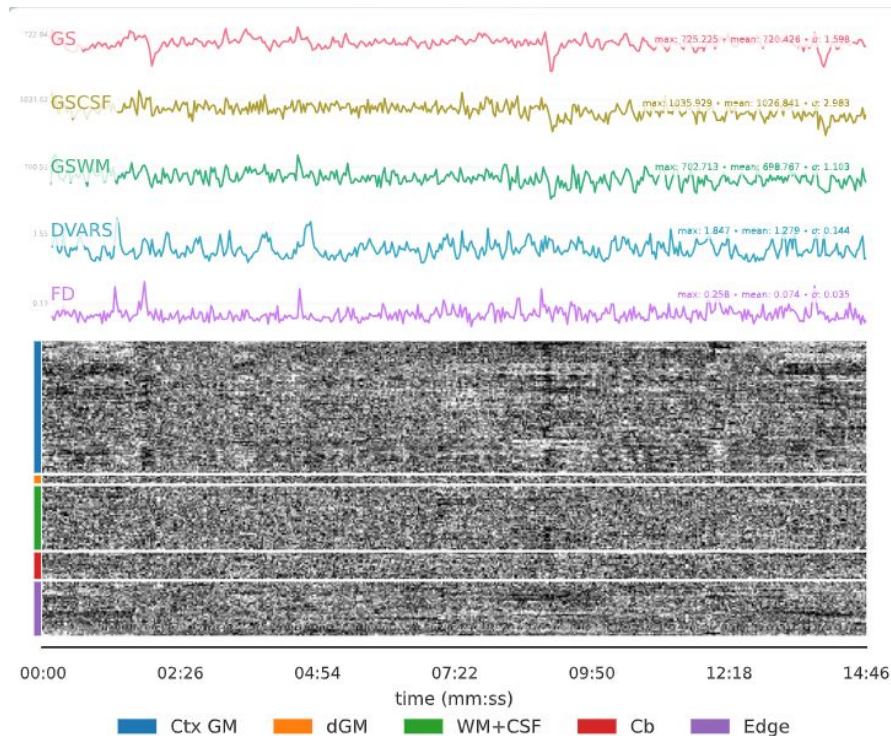
Get figure file: [sub-038/figures/sub-038_ses-02_task-WPT_run-02_desc-flirtnobbr_bold.svg](#)

fMRIPrep output (1): Visual QA Reports

Head rotation, spikes



Relatively clean subject



fMRIPrep output (2): Derivatives

Anatomical output (/anat)

Main files:

`desc-preproc_T1w.nii.gz`
`desc-brain_mask.nii.gz`
`dseg.nii.gz` (tissue segmentation)
`probseg.nii.gz` (GM / WM / CSF probabilities)

What they represent:

- Skull-stripped, bias-corrected T1 image
- Brain mask
- Tissue classification

Used for:

- Registration
- ROI definition
- Structural analyses

Functional output (/func)

Main files:

`sub-001_ses-01_task-NF_run-04_`
`space-MNI152NLin2009cAsym_res-2_`
`desc-preproc_bold.nii`

`sub-001_ses-02_task-WPT_run-01_`
`space-MNI152NLin2009cAsym_res-2_`
`Desc-preproc_bold.nii`

Properties:

- Motion-corrected
- Slice-time corrected (if enabled)
- Distortion corrected (if fieldmaps available)
- Aligned to T1w (native) and/or standard space (MNI template)

fMRIPrep output (3): Confounds.tsv

fMRIPrep estimates confounds. You decide which to use in your analyses!

- Motion regressors: `trans_x/y/z`, `rot_x/y/z` (+ `derivs` & `powers`)
→ Always include at least standard six parameters
- Global Signal: `global_signal`
→ Debated: explicit choice required
- Physiological: `white_matter`, `csf`
→ Standard inclusion
- CompCor: `a_comp_cor_00-05`, `t_comp_cor_00-05`
→ aCompCor for noise
- Scrubbing: `framewise_displacement`, `std_dvars`
→ $FD > 0.5$ mm → flag/remove

Note:

fMRIPrep
doesn't perform
any denoising
itself!

Parallel processing

One subject at a time may not be fast enough ...

- Typical fMRIPrep runtime: 1-8 hours per subject
- 80 subjects sequential = up to 640 hours = 26 days!



Solution: run multiple subjects in parallel (on a cluster)

	Local machine	HPC cluster	Cloud
Tool	<code>--nprocs</code> flag	SLURM / PBS job arrays	AWS / Google Cloud
Best for	Small N, testing	Large datasets, institutional	Scalable, expensive
Complexity	Low	Medium	High

Parallel processing: The setup

We create a script that runs multiple subjects at once.

We also tell docker how much memory and CPU cores to use for each subject.

```
cd C:\Users\<Your username>
```

```
notepad .wslconfig
```

We will run three subs in parallel, each using 8gb memory and 4 cores.

So wsl needs 24gb and 24 cores in total:

```
[wsl2]
```

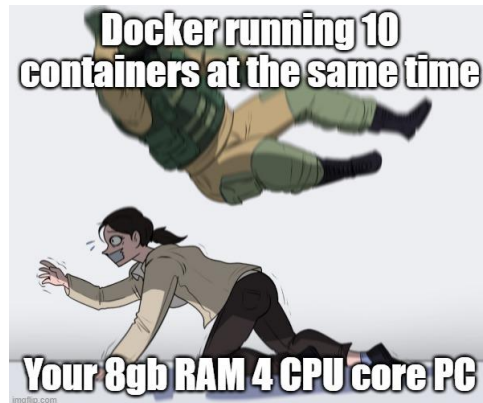
```
memory=24GB
```

```
processors=16
```

```
swap=2GB
```

Stick to 8gb memory per sub and at least 4 cores.

If your device is weaker, run fewer subjects.



Parallel processing: The script

Now adjust your `run_fmriprep.sh` script, see notebook.

Most important part `-NPROCS` tells docker to use 4 cores per sub, and `MEM_MB` 8gb memory.

`N_jobs / N` determines how many subjects are done in parallel. If you want to run more or less subs, adjust `.wslconfig` accordingly.

HeuDiConv: handling multi-session studies

BIDS session structure

→ `sub-01/ses-01/`, `sub-01/ses-02/`

- Ensure session detection/identification
 - separate DICOM folders *or* define sessions manually
- Update heuristic file (heuristic.py)
 - include `_ses-{session}` in all filenames/paths
- Specify sessions in bash command (run_heudiconv.sh)
 - add `-ss` (e.g., `-ss 01 02`)
- Adjust data template (`-d`)
 - include `{session}` if needed

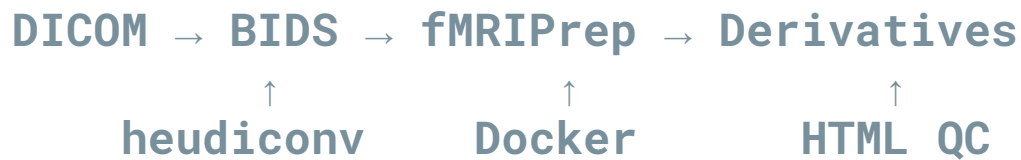
```
# Get the subject and session from environment variables
subject = os.getenv("SUBJECT_ID")
session = os.getenv("SESSION_NUM")
```



HIPPNF_VP_001_01
HIPPNF_VP_001_02
HIPPNF_VP_002_01
HIPPNF_VP_002_02
HIPPNF_VP_003_01
HIPPNF_VP_003_02
HIPPNF_VP_005_01
HIPPNF_VP_005_02

```
data = create_key(f"sub-{subject}/ses-{session}/anat/sub-{subject}_ses-{session}_T1w")
```

fMRIPrep is the beginning, not the end



Derivatives folder:

- [sub-XX/anat/](#) — preprocessed T1w, brain mask, segmentation
- [sub-XX/func/](#) — preprocessed BOLD in multiple spaces, brain mask
- [sub-XX/func/*_confounds_timeseries.tsv](#) — motion, CompCor, FD, DVARs
- [sub-XX.html](#) — visual QC report, check this for every subject

Question: what is fMRIPrep's “minimal processing” philosophy?

fMRIPrep is the beginning, not the end

DICOM → BIDS → fMRIPrep → [confound selection] → [your analysis]

Task fMRI (contrasts) / GLM:



- FSL FEAT – GUI-based, widely used, good documentation
- SPM – MATLAB-based, very widely used in clinical neuroimaging
- nilearn – Python, flexible, scriptable, integrates well with fMRIPrep output
- fitlins – BIDS-native GLM tool, uses model JSON files

Resting-state connectivity:

- nilearn – seed-based and ICA connectivity
- CONN toolbox – MATLAB, comprehensive, widely used
- XCP-D – BIDS-native postprocessing pipeline built on fMRIPrep output

MVPA / decoding:

- nilearn – Decoder, SearchLight
- PyMVPA – older but comprehensive

Representational similarity analysis (RSA):

- rsatoolbox (Python)
- Custom implementation in Python/MATLAB

fMRIPrep is the beginning, not the end

DICOM → BIDS → fMRIPrep → [confound selection] → [your analysis]



Confound handling is not one-size-fits-all.

Task-GLM

BOLD ~ task + motion +
physiological noise

Smoothed (4-8 mm FWHM)
Include motion (6-24 params)
Include aCompCor
Optional: scrubbing
High-pass filtering (100-200s)

MVPA / decoding

Decode patterns, not mean
signal

Avoid aggressive denoising
Smoothing (0-4 mm FWHM)
Only light motion regression

Resting state

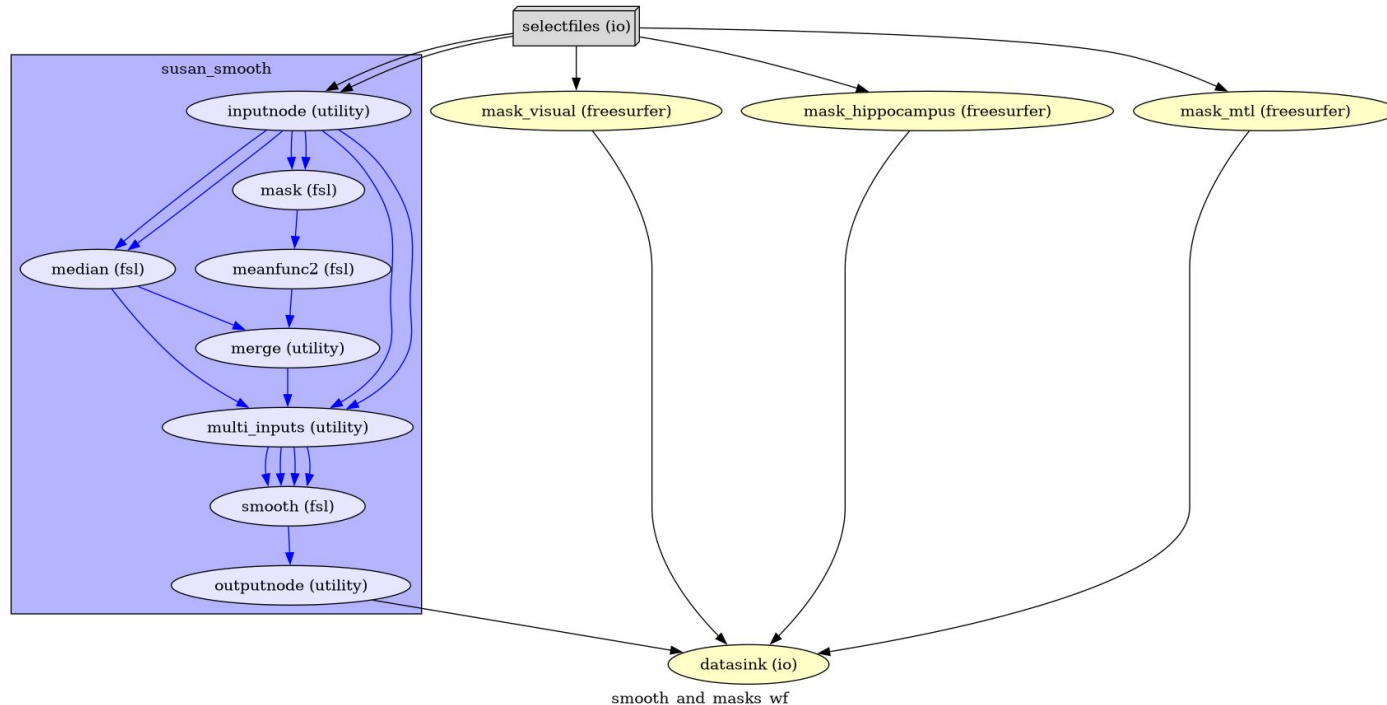
Seed-based conn, ICA,
dynamic conn, graphs/nets

Motion regression (6-24)
Framewise Displacement
aCompCor (WM, CSF)
Glob signal (controversial)
Temporal filtering (0.01-0.1 Hz)
Moderate smoothing (4-6mm)

fMRIPrep is the beginning, not the end

Working with NiType: workflow engine for neuroimaging

Analyses involve many steps that are dependent on each other



fMRIPrep is the beginning, not the end

Working with fMRIPrep outputs in SPM: unzipping

fMRIPrep stores functional images as compressed .nii.gz files.

However, SPM cannot read .gz files directly.

Key step before analysis: You need to **unzip** all .nii.gz files to .nii format.

Now that your file structure is BIDS conform, it's super easy to loop through:
All subjects, sessions, and func folders, to find all .gz files (per task and run).

Then: use **gunzip** to extract them automatically.

Recommendations for reproducibility and sharing

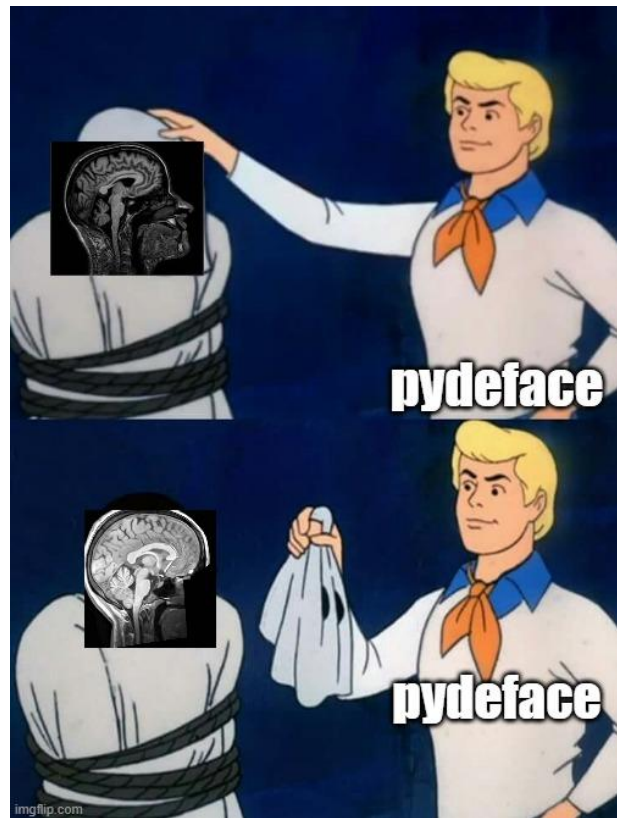
Defacing

We will use the poldracklab/pydeface docker container

Now it's your turn!

Try to setup a bash script to deface all T1w's.

Save them under [./preprocessed/sub-*/defaced](#)



Recommendations for reproducibility and sharing

Record and report in your methods:

- fMRIPrep version (results can differ between versions!)
- FreeSurfer version
- Output spaces (i.e., template)
- Confound strategy
- Scrubbing threshold
- Smoothing kernel
- fMRIPrep command (i.e., flags)
- References!

→ DataLad: version-control and
Sharing of large fMRI datasets

Methods

We kindly ask to report results preprocessed with this tool using the following boilerplate.

HTML Markdown LaTeX

Results included in this manuscript come from preprocessing performed using *fMRIPrep* 20.2.3 (Esteban, Markiewicz, et al. (2018); Esteban, Blair, et al. (2018); RRID:SCR_0016216), which is based on *Nipype* 1.6.1 (Gorgolewski et al. (2011); Gorgolewski et al. (2018); RRID:SCR_002502).

Anatomical data preprocessing

A total of 1 T1-weighted (T1w) images were found within the input BIDS dataset. The T1-weighted (T1w) image was corrected for intensity non-uniformity (INU) with `n4biasfieldcorrection` (Tustison et al. 2010), distributed with ANTs 2.3.3 (Avants et al. 2008; RRID:SCR_004757), and used as T1w-reference throughout the workflow. The T1w-reference was then skull-stripped with a *Nipype* implementation of the `ants88lasf1d0correction` workflow (from ANTs), using OASIS30ANTs as target template. Brain tissue segmentation of cerebrospinal fluid (CSF), white-matter (WM) and gray-matter (GM) was performed on the brain-extracted T1w using `fast` (FSL 5.0.9; RRID:SCR_002823; Zhang, Brady, and Smith 2001). Brain surfaces were reconstructed using `recon-all` (FreeSurfer 6.0.1; RRID:SCR_001847; Dale, Fischl, and Sereno 1999), and the brain mask estimated previously was refined with a custom variation of the method to reconcile ANTs-derived and FreeSurfer-derived segmentations of the cortical gray-matter of Mindboggle (RRID:SCR_002438; Klein et al. 2017). Volume-based spatial normalization to one standard space (MN152NLin2009cAsym) was performed through nonlinear registration with `antsRegistration` (ANTs 2.3.3), using brain-extracted versions of both T1w reference and the T1w template. The following template was selected for spatial normalization: *ICBM 152 Nonlinear Asymmetrical template version 2009c* (Fonov et al. (2009); RRID:SCR_008796; TemplateFlow ID: MN152NLin2009cAsym).

Functional data preprocessing

For each of the 3 BOLD runs found per subject (across all tasks and sessions), the following preprocessing was performed. First, a reference volume and its skull-stripped version were generated using a custom methodology of *fMRIPrep*. Susceptibility distortion correction (SDC) was omitted. The BOLD reference was then co-registered to the T1w reference using `bbregister` (FreeSurfer) which implements boundary-based registration (Greve and Fischl 2009). Co-registration was configured with six degrees of freedom. Head-motion parameters with respect to the BOLD reference (transformation matrices, and six corresponding rotation and translation parameters) are estimated before any spatiotemporal filtering using `mcflirt` (FSL 5.0.9; Jenkinson et al. 2002). The BOLD time-series (including slice-timing correction when applied) were resampled onto their original, native space by applying the transforms to correct for head-motion. These resampled BOLD time-series will be referred to as *preprocessed BOLD in original space*, or just *preprocessed BOLD*. The BOLD time-series were resampled into standard space, generating a *preprocessed BOLD run* in MN152NLin2009cAsym space. First, a reference volume and its skull-stripped version were generated using a custom methodology of *fMRIPrep*. Several confounding time-series were calculated based on the *preprocessed BOLD*: framewise displacement (FD), DVARS and three region-wise global signals. FD was computed using two formulations following Power (absolute sum of relative motions, Power et al. (2014)) and Jenkinson (relative root mean square displacement between affines, Jenkinson et al. (2002)). FD and DVARS are calculated for each functional run, both using their implementations in *Nipype* (following the definitions by Power et al. 2014). The three global signals are extracted within the CSF, the WM, and the whole-brain masks. Additionally, a set of physiological regressors were extracted to allow for component-based noise correction (CompCor; Behzadi et al. 2007). Principal components are estimated after high-pass filtering the *preprocessed BOLD* time-series (using a discrete cosine filter with 128s cut-off) for the two CompCor variants: temporal (tCompCor) and anatomical (aCompCor). tCompCor components are then calculated from the top 2% variable voxels within the brain mask. For aCompCor, three probabilistic masks (CSF, WM and combined CSF+WM) are generated in anatomical space. The implementation differs from that of Behzadi et al. in that instead of eroding the masks by 2 pixels on BOLD space, the aCompCor masks are subtracted a mask of pixels that likely contain a volume fraction of GM. This mask is obtained by dilating a GM mask extracted from the FreeSurfer's `aseg` segmentation, and it ensures components are not extracted from voxels containing a minimal fraction of GM. Finally, these masks are resampled into BOLD space and binarized by thresholding at 0.99 (as in the original implementation). Components are also calculated separately within the WM and CSF masks. For each CompCor decomposition, the *k* components with the largest singular values are retained, such that the retained components' time series are sufficient to explain 50 percent of variance across the nuisance mask (CSF, WM, combined, or temporal). The remaining components are dropped from consideration. The head-motion estimates calculated in the correction step were also placed within the corresponding confounds file. The confound time series derived from head motion estimates and global signals were expanded with the inclusion of temporal derivatives and quadratic terms for each (Satterthwaite et al. 2013). Frames that exceeded a threshold of 0.5 mm FD or 1.5 standardised DVARS were annotated as motion outliers. All resamplings can be performed with a *single interpolation step* by composing all the pertinent transformations (i.e. head-motion transform matrices, susceptibility distortion correction when available, and co-registrations to anatomical and output spaces). Gridded (volumetric) resamplings were performed using `antsApplyTransforms` (ANTs), configured with Lanczos interpolation to minimize the error (other kernels (Lanczos 1964). Non-gridded (surface) resamplings were performed using `resample-volume` (FreeSurfer).

However ... some critical things to remember

- **fMRIPrep is minimal by design** - it stops where your scientific decisions begin
→ e.g., don't apply maximal smoothing before MPVA
- **Always check the HTML report** - for every subject, before any analysis
→ don't wait to discover problems until after the analysis! (output folders $\neq$ success)
- **Test on 2–3 subjects before scaling up** - pin to a specific version (:latest Docker tag)
→ don't mix different fMRIPrep versions across subjects in the same dataset
- **Using --work-dir to write temporary files to a partition with plenty of space**
→ in general: running fmriprep from local folders (rather than a server) saves lots of time!
- **Don't blindly trust the pipeline!** - use fmriprep because you understand preprocessing, not instead of it

When fMRIPrep may not be the right tool

- **Non-standard populations.** fMRIPrep is optimized for healthy adults. Clinical populations with significant atrophy or lesions or very young children (infants) may require dedicated pipelines!
- **Reproducing an existing in-house pipeline or results.** if you want to specifically reproduce certain results, it may be better to use the exact same preprocessing steps/parameters as previously used, even when fMRIPrep does the same things on a conceptual level.
- **Learning purposes.** if the goal is to understand each preprocessing step deeply, or to learn shell scripting and MATLAB/Python in more depth, building a pipeline manually has real pedagogic value.

Nearing the end... Any questions?



Docs

fmripred.org/en/stable



Forum

<https://neurostars.org/>
(Tag: fmripred)



Bugs

github.com/nipreps/fmripred



BIDS

bids.neuroimaging.io



Slides

[sterrevidl/
fmripred-workshop](https://sterrevidl/fmripred-workshop)



Paper

[https://doi.org/10.1038/
s41592-018-0235-4](https://doi.org/10.1038/s41592-018-0235-4)

Concluding remarks: what we covered ...

Day 1:

- Why fMRIPrep and the minimal preprocessing philosophy
- Installation process (WSL, Docker)
- DICOM → BIDS with heudiconv
- Running fMRIPrep

Day 2:

- fMRIPrep outputs
- Parallelizing across subjects locally and on HPC
- What analysis tools to use next

But also, what fMRIPrep is not:

- A black box you can trust blindly
- The right tool for every dataset and every population
- A replacement for understanding what preprocessing actually does